Autonomous Cognitive Engine – Project Report

Project Title: Autonomous Cognitive Engine for Deep Research and Long-Horizon Tasks

Abstract

The Autonomous Cognitive Engine (ACE) is a modular, tool-governed AI system built to execute complex, multi-step objectives with deterministic behavior, memory safety, and auditable reasoning. Unlike stateless conversational agents, ACE uses a Supervisor Agent to orchestrate specialized sub-agents (Research, Summarization, etc.), a Virtual File System (VFS) for durable memory, and a strict tool-gating policy that prevents unauthorized web access and implicit reasoning. The system integrates LangChain, DeepAgents architecture, Groq LLMs, and LangSmith observability to deliver reproducible, testable long-horizon workflows for tasks such as market research, strategic planning, and analytic reporting.

1. Project Introduction

Modern real-world tasks such as market research and strategy planning require decomposition into multiple sub-tasks, durable state, and auditable execution. ACE addresses these requirements by combining a Supervisor Agent that enforces a deterministic, single-tool-per-turn policy with specialized sub-agents and an explicit Virtual File System (VFS). The architecture prevents hallucinations by forcing tool-grounded cognition, ensures reproducibility with persistent artifacts, and logs every action using LangSmith for observability.

Primary goals

- Reliable long-horizon reasoning and execution
- Strict tool governance and role isolation
- Persistent, auditable external memory (VFS)
- Reproducible workflows with strong testing coverage

---

2. Tools & Frameworks Used

- **LangChain** — orchestration of prompts and tool interfaces
- **DeepAgents architecture** — multi-agent design patterns and isolation
- **Groq LLM** — core language model used for reasoning and generation
- **LangSmith** — observability (logging prompts, tool calls, traces)

---

3. Overall Architecture

A single Supervisor Agent is the control plane. It receives user intent, decomposes tasks, and invokes one sub-agent or tool per turn. Sub-agents are strictly permissioned:

- **Supervisor Agent** — planning, policy enforcement, tool registry
- **Research Sub-Agent** — exclusively responsible for web queries (sandboxed)
- **Summarization Sub-Agent** — transforms long text into concise structures
- **VFS** — persistent storage for artifacts (notes, charts, CSVs)
- **Calendar Module** — JSON-schema validated scheduling operations
- **Testing Suite** — unit, integration, and scenario tests logged to LangSmith

---

4. Project Modules

4.1 Supervisor Agent (main/app.py)

Role: Central authority that interprets user intent, selects tools/sub-agents, enforces a strict system prompt, and records every decision to LangSmith.

Responsibilities

- Load environment and credentials
- Initialize Groq LLM and chain-of-tools
- Register tools with strict capability metadata
- Enforce single-tool-per-turn rule
- Validate tool outputs and persist to VFS when required
- Maintain execution trace in LangSmith

Example pseudocode

```
# main/app.py (pseudocode)
from agents import Supervisor
from tools import registry, VFS, LangSmithClient


def main():
    env = load_env(".env")
    llm = init_groq_llm(env['GROQ_KEY'])
    supervisor = Supervisor(llm=llm, tools=registry(), vfs=VFS(), logger=LangSmithClient())
    supervisor.serve()
```

4.2 Virtual File System (VFS) (memory/vfs.py)

Role: Durable external memory for storing intermediate outputs, datasets, summaries and audit logs—avoids relying on model context.

Key features

- File read/write with versioning and metadata
- Searchable index for retrieval (by tags, timestamps, provenance)
- Audit trail linking files to agent actions and LangSmith traces
- Structured artifact support (JSON, CSV, PNG, Markdown)

Example interface

```
# memory/vfs.py (interface)

def write_file(path: str, content: bytes, metadata: dict) -> str: ...

def read_file(path: str) -> Tuple[bytes, dict]: ...

def list_files(prefix: str=None) -> List[str]: ...
```

4.3 Calendar Management Module (tools/calendar_tools.py)

Design: All events represented as JSON objects validated against a schema (title, start, end, timezone, metadata). No free-text scheduling.

Advantages

- Deterministic scheduling, machine-readable, audit & reversal

Schema (example)

```
{
  "title": "Project Sync",
  "start": "2025-01-21T10:00:00+05:30",
  "end": "2025-01-21T11:00:00+05:30",
  "tz": "Asia/Kolkata",
  "metadata": {"location": "Zoom", "attendees": ["a@uni.edu"]}
}
```

4.4 Controlled Web Search Module (tools/web_search.py)

Design guarantees

- Only the Research Sub-Agent may call this module
- Responses are source-validated and returned with provenance metadata

- Safety filters and whitelist/blacklist rules are applied before data enters VFS

Returned structure


```
{
  "query": "market size for X in 2024",
  "results": [
    {"url":"...", "title":"...", "snippet":"...", "retrieved_at":"...","trust_score":0.9}
  ]
}
```

4.5 Research Sub-Agent

Characteristics

- Sole agent with web access
- Restricted toolset (web_search, doi_lookup, dataset_fetch)
- Cannot write to VFS — returns structured output to Supervisor
- Responsible for source-grounded facts and citation metadata

Typical flow

1. Supervisor sends research intent & constraints
2. Research agent performs sandboxed web queries
3. Returns structured findings + provenance for Supervisor validation

---

4.6 Summarization Sub-Agent

Characteristics

- Takes only provided text (or VFS file contents supplied by Supervisor)
- Produces multi-level summaries (TL;DR, structured bullets, references)
- No access to web, VFS write, or scheduling tools
- Tuned for compression, clarity, and hierarchical output

Output example


```
{ "tldr": "Two-line summary", "bullets": ["point1","point2"],
"sections":[{"title":"Methods","text":"..."}] }
```

4.7 Testing Strategy

Objectives

- Verify strict tool-gating and role isolation
- Prevent hallucinations by confirming all claims have source artifacts
- Ensure VFS integrity and state consistency across multi-step workflows

Unit test example

```
def test_vfs_write_read():
  write_file("a.txt","hello", {"author":"test"})
  assert read_file("a.txt").content == b"hello"
```

Integration / scenario tests

- Research → Supervisor validates → Supervisor writes to VFS → Summarization invoked.
- Long-horizon planning where tasks spawn subtasks and rely on VFS artifacts

---

5. Current Progress

- Full Supervisor Agent prototype able to plan and delegate tasks
- VFS implemented with read/write and metadata versioning
- Research Sub-Agent sandboxed web search working with provenance capture
- Summarization Sub-Agent tested for hierarchical compression
- LangSmith integrated for logging prompts, tool calls, and traceability
- Automated test suite covers core unit and integration flows

Status: Production-grade prototype with enforced role isolation and auditable execution. Some modules (e.g., dynamic agent scaling, multi-modal ingestion) remain as roadmap items.

---

6. Challenges

1. **System complexity & maintenance** — strict separation increases development overhead and debugging complexity.
2. **Latency** — multi-step tasks (web calls, I/O, rendering) can cause visible delays. Needs asynchronous management and progress reporting.
3. **Cost management** — multiple LLM calls and logging increase token & infra costs; requires caching, batching, and selective logging.

---

7. Future Scope

1. **Multi-Modal Capabilities** — ingest PDFs, images, spreadsheets, and audio while maintaining strict tool policies.
2. **Dynamic Agent Scaling** — spawn domain-specific agents on demand and auto-retire them after task completion.
3. **Advanced Planning & Self-Reflection** — DAG-based long-term planning graphs, self-evaluation loops, and confidence scoring to allow autonomous re-planning.

Env / run

- .env contains keys for GROQ, LangSmith, and any other services
- pip install -r requirements.txt
- python src/main/app.py --config configs/prod.yml

---

8. Experiments & Evaluation (Template)

Setup

- Hardware: CPU/GPU details, memory, disk
- Software: Python version, library versions, commit hash

Metrics

- Task success rate (end-to-end)
- Precision of extracted facts (via source checks)
- Latency (per task and total)
- Cost (tokens / compute)

---

9. Appendices

Installation & Run Guide

1. Clone repo and switch to branch:
2. git clone <repo-url>
3. cd Autonomous-Cognitive-Engine-for-Deep-Research-and-Long-Horizon-Tasks
4. git checkout intern-arunika
5. Create virtualenv and install:
6. python -m venv .venv
7. source .venv/bin/activate
8. pip install -r requirements.txt

9. Provide .env with GROQ_KEY, LANGSMITH_KEY, etc.
10. Run Supervisor: python src/main/app.py --config configs/local.yml